Sheetwright

Native Google Sheets for Claude

User manual & setup guide

Edit your Google Sheets with real formatting, links to Drive and native Tables — without converting to Excel.

1. Why this exists

Many people organize their world in spreadsheets: invoice lists, budget comparisons, trackers, inventories. It is one of the most elegant ways to handle information without setting up a database: each row indexes something and, with a link, opens the real document. The richness lives in the details — the colors that separate statuses, the currency format, the cells that link to a PDF in Drive.

The problem: until now, when an AI assistant read a Google Sheet, it saw a flat CSV. Colors, visual order and links were lost. Existing integrations read and write values, but not the formatting that makes a spreadsheet usable.

Sheetwright solves that. **It gives Claude official access (through Google's API) to work your spreadsheets natively: it writes directly on the real sheet, with real formatting, HYPERLINK formulas that open your files, and native Tables — without converting anything to Excel or back.**

2. What it lets you do

- **Real formatting:** color cells and headers, bold, number and currency formats (respecting your regional settings).
- **Links to Drive:** HYPERLINK formulas that open invoices, receipts or any of your Drive files right from a cell.
- **Native Tables:** Google Sheets Tables, with column types, dropdown menus and built-in filters.
- **Create spreadsheets:** generate new sheets in your Drive, already formatted.
- **Organize files:** move and tidy your Drive files into whatever folders you want.
- **Locale-aware:** it adapts to each sheet's regional settings, respects them, and flags inconsistencies — offering to fix them with your OK.

So you can ask any Claude session, in plain language: *"fill in this comparison with the quotes I got by email and tidy the files into that folder."*

3. How it works (in plain terms)

Claude runs in the cloud, not on your computer. To work your spreadsheets it uses two distinct paths that are worth not confusing:

The two "bridges"

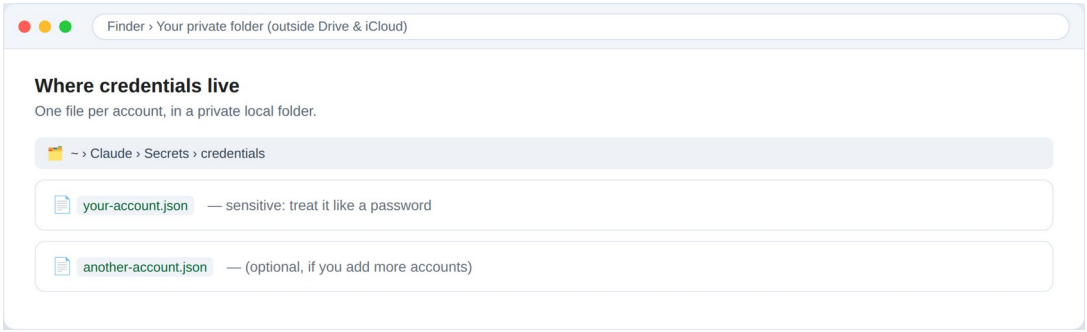
- **API bridge (the subject of this manual):** programmatic access to Google Sheets and Drive. With this, Claude edits your spreadsheets natively. It works even if your computer is off, because it uses a stored permission.
- **File bridge:** when you connect a folder from your computer to Claude. Good for loose files (PDFs, images). Requires the desktop app to be open.

> **Key idea:** the API bridge gives you access to ALL your spreadsheets and files in that Google account, no matter which folder they are in. You don't need to "mount" anything to edit your Sheets.

Access is configured once per Google account, and stored in a private credentials file that lives on your computer (never in the cloud, never in Drive).

4. Where the credentials live (and why it matters)

Access is stored in one file per account (for example, your-account.json). That file is sensitive: it is equivalent to a password, because it can read and write your Sheets and your Drive. That is why it is kept in a private, local folder on your computer, outside Google Drive and iCloud, so it never syncs to any cloud.



The credentials live in a private local folder — one per account.

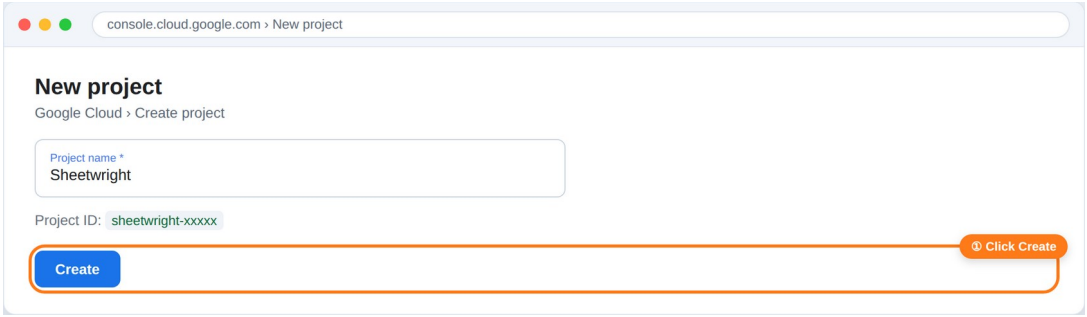
> You can revoke access anytime **at myaccount.google.com/permissions**. And if you need Claude to work without your computer open (scheduled tasks), the alternative is to store the permission as an environment variable in the execution environment, instead of a file.

5. First-time setup (once per account)

It takes about 15 minutes. You do it once per Google account and it lasts forever. You need your Google account and a browser. A Claude assistant can guide you — or do the clicks for you if you let it drive the browser — but here is the full walkthrough so you know what happens at each step.

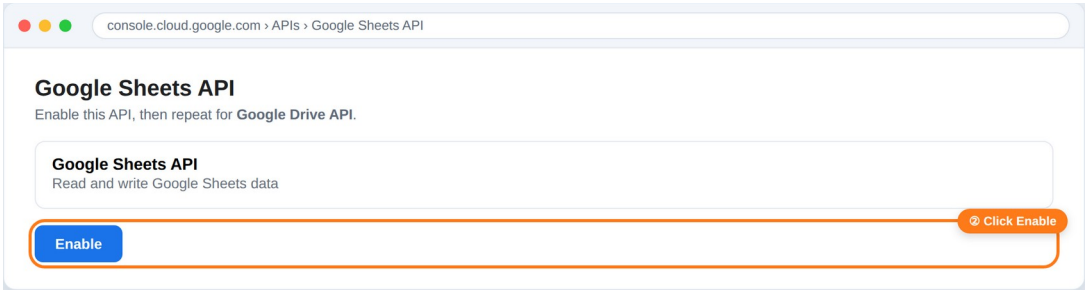
Step 1 — Create a project in Google Cloud

Go to console.cloud.google.com, create a new project and give it a recognizable name (for example, "Sheetwright").



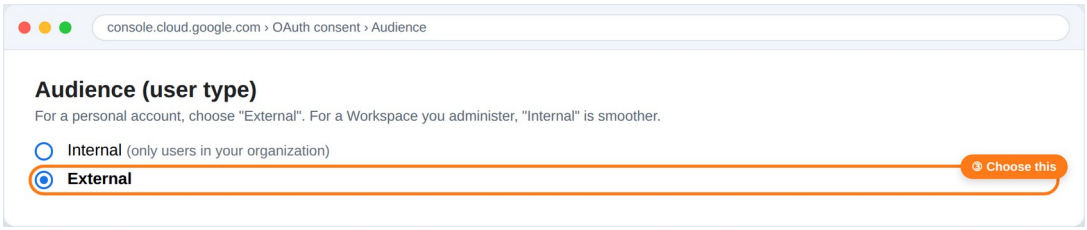
Step 2 — Enable the Sheets and Drive APIs

In the project, enable the "Google Sheets API" and the "Google Drive API" (there are two; they are enabled the same way).



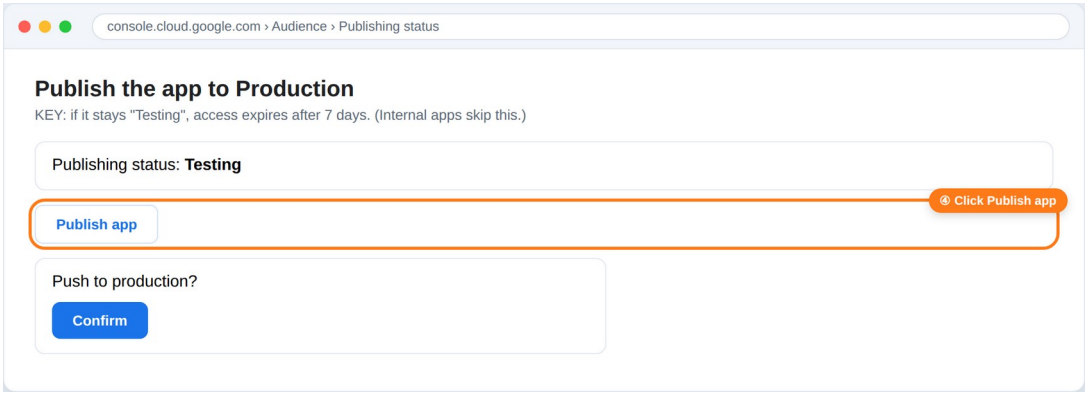
Step 3 — Configure consent and pick the user type

On the consent screen (Google Auth Platform), fill in the app name and your email. If it is a personal account, choose the “External” user type. If it is a Google Workspace account you administer, choose “Internal” — no “unverified app” warning, no publishing step, and the permission doesn’t expire.



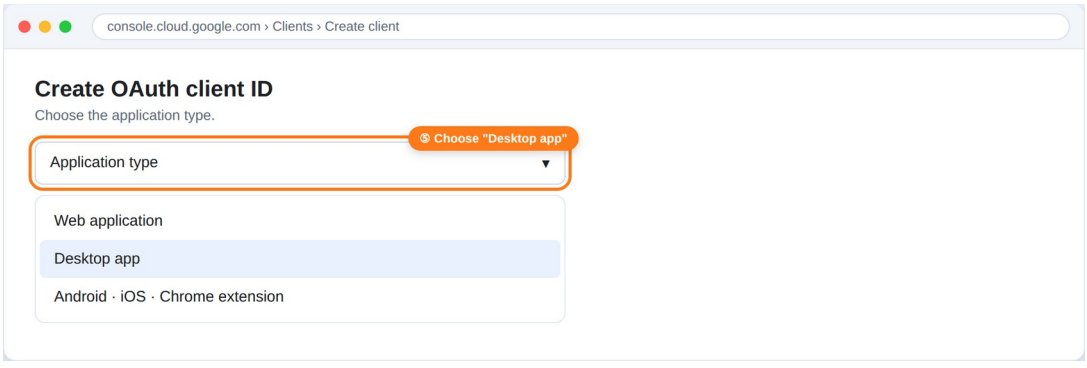
Step 4 — Publish the app to Production (External accounts only)

> **This step is key:** if an External app stays in “Testing” mode, access expires after 7 days and you would have to reconfigure every week. Publishing it to Production means the permission doesn’t expire. (Internal apps skip this entirely.)



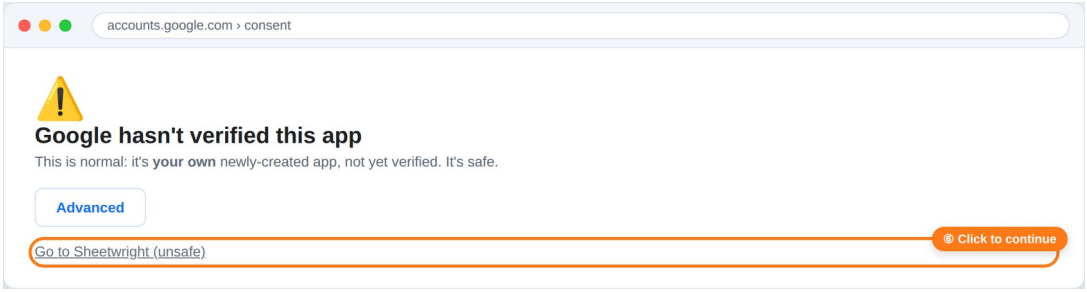
Step 5 — Create the credential (Desktop app)

Create an “OAuth client ID” of type “Desktop app”. Google will show you an identifier (client ID) and a secret (client secret): save them, because the secret is only shown when created.

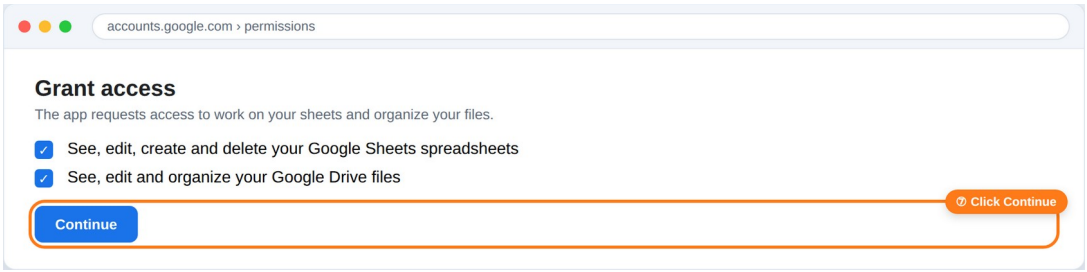


Step 6 — Authorize access in the browser

With that credential, Claude opens a Google page for you to authorize. For External/personal apps you will see an "app not verified" notice: this is normal and safe, because it is your own freshly-created app. Click "Advanced" and continue. (Internal apps don't show this.)



Then the permissions screen appears. Grant access to Sheets and Drive, and click Continue.



> **A technical detail:** at the end, the browser takes you to a page that "doesn't load" (it says localhost). That is expected — the code Claude needs to finish is right there in the address bar. It is not an error.

Step 7 — Save the credentials

Claude saves the your-account.json file in your private local folder (for example ~/Claude/Secrets/credentials/). Done: the bridge is set up for that account, for good.

6. Day-to-day use

Once it is set up, you don't have to think about anything technical. You talk to any Claude session in natural language. Some examples:

- "Fill in this budget comparison with the quotes I got by email and color the cheapest one green."
- "Build an invoice list with a column that opens each one from Drive."
- "Turn this range into a native Table with a Status dropdown."
- "Create a new spreadsheet in my Drive and format it like this."

If you have more than one account configured, just say which one: *"...in the spreadsheet on such-and-such account."*

> **About the browser:** Sheetwright does its work through the API silently — it does not need to open a browser. If you ever want to see a change with your own eyes, you can ask Claude to open the sheet to verify; it will tell you before doing so. By default, nothing pops up.

7. Locale & languages (why your sheets don't break)

A Google Sheet carries its own regional settings (locale) and timezone, stored inside the document. Those two fields — and only those — decide how formulas are parsed (the argument separator is "," in some regions and ";" in others) and how currency, numbers and dates are formatted by default.

Sheetwright reads those fields through the API and adapts to them. What it deliberately ignores are the "display" layers that have no effect on the data: your operating system language, your Google account

language, the Sheets menu language, and your browser language. You can have Chrome in English, your account in English, the menus in Spanish and the document set to Spain — it doesn't matter. Sheetwright acts on the document's locale, which is why it is robust to any odd combination.

Respect, flag, and offer to fix

- **Respect:** it never changes an existing sheet's locale silently.
- **Flag once:** if a sheet's regional settings look inconsistent with your context (a common real case: a new sheet born in "Spain / Euro / Madrid" when you are in Argentina), it says so once — that mention in the reply is the flag.
- **Offer options:** use your local currency and format without touching the document, leave everything as is, or fix this sheet's configuration (with your OK) — now or later.

> **The recurring case that isn't an API fix:** if ALL your new sheets are born with the wrong locale, that is a Google account setting, not something the API can change. Google's official help explains how to change it: support.google.com/docs/answer/58515 (and per sheet: File → Settings → Locale / Time zone).

8. Usage variants (for sharing with others)

The same approach scales from one account to many, and from one person to a team. Three decisions worth knowing:

One skill, many accounts

The logic (the "how") is packaged once as a reusable skill, with no secret inside. The credentials (the "who") live separately, one file per account. Adding a new account is just adding another file; the skill isn't touched. Another person installs the same skill and runs setup with their own account.

Own project vs. shared project

- **One project per person:** each person creates their own Google Cloud project. Fully independent — ideal for sharing with outside people.
- **Shared project:** a single project that several people authorize. Simpler onboarding (they skip creating the project), but everyone depends on that project. Good for a team.

Local vs. unattended

- **Local (recommended):** credentials in a private folder on the computer. Maximum privacy; requires the desktop app open when used.
- **Unattended:** the permission as an environment variable, so it can run on its own (scheduled tasks) without the computer open.

9. Security & hygiene

- **Treat the credentials file like a password:** never upload it to Drive, a synced folder, or a repository.
- **Revoke anytime:** myaccount.google.com/permissions.
- **If a permission was exposed, rotate:** revoke the old permission first and then mint a new one. Order matters (see note).

> **About rotation:** when you revoke a permission, Google may invalidate all of that app's permissions at once. That is why it is best to revoke first and only then mint the new one — so the new one stays alive.

10. Common problems

- **Links show #ERROR!:** it is a regional-settings issue. In comma-decimal locales, formulas use ";" instead of ",". Sheetwright already detects this and handles it on its own.

- **Access “stopped working” after a week:** the app stayed in “Testing” mode. Publish it to Production (Step 4), or use the Internal user type on a Workspace account.
- **“It can’t find the credentials”:** the desktop app has to be open and the private folder connected, so Claude can read that account’s file.
- **Currency format “gets lost” in a table:** in native Tables you must use the “Currency” column type (the column type overrides any manual format).

Author, support & license

Created by **Miguel Angel Haluza** (github.com/mahaluza). Questions, bugs or feedback: open an issue at github.com/mahaluza/sheetwright/issues.

In one sentence

You set up access once, keep the permission in a private file on your computer, and from then on you talk to Claude in plain language so it works your Google Sheets the way you would — with colors, links and tables — without converting anything to Excel.